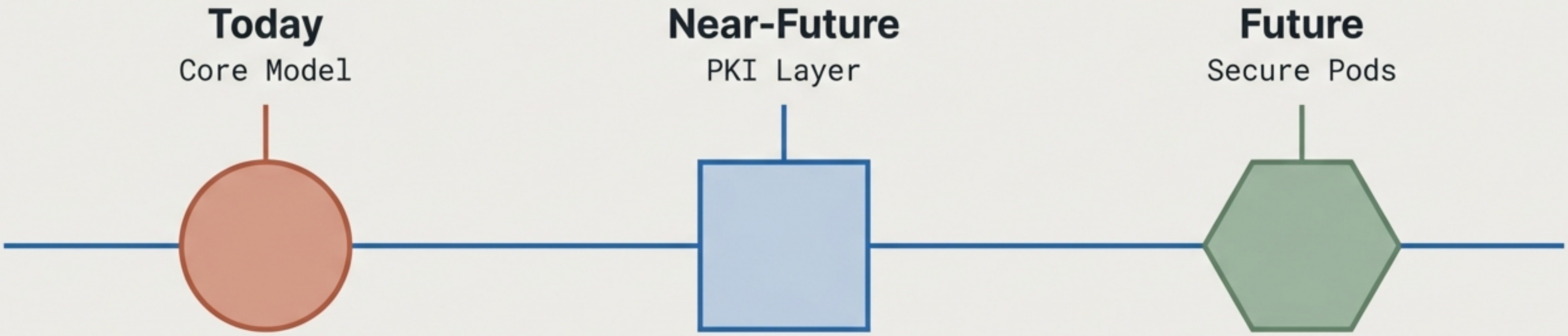
SG/Send Architecture Evolution

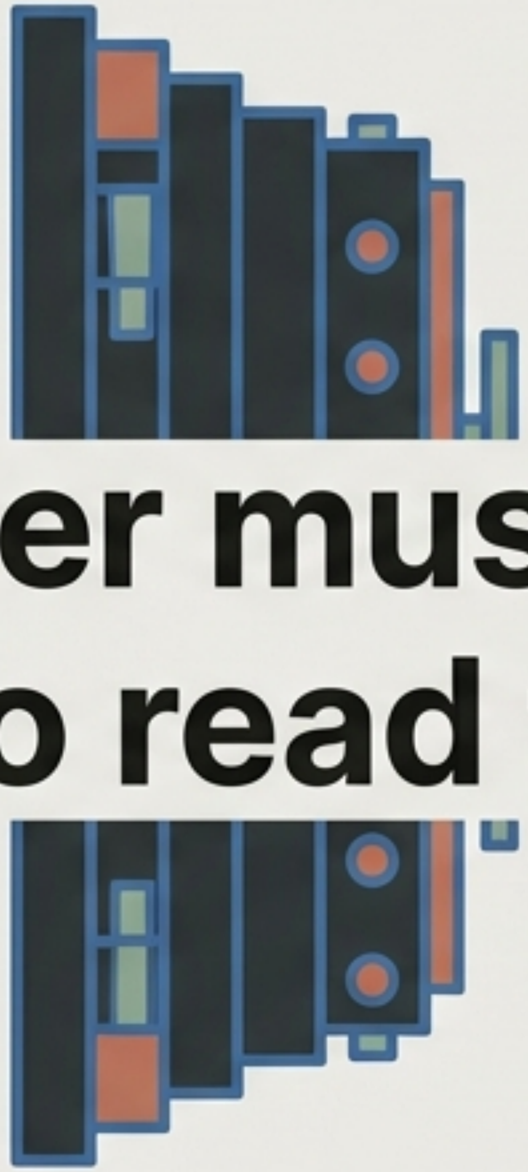
Document Version: v0.15.5
Date: March 2026
Authors: Dinis + the SG/Send team



This presentation acts as three architecture snapshots detailing the progression of zero-knowledge file sharing from foundational principles to advanced encrypted compute environments.

The uncompromising principle driving every design decision

SERVER



The server must never be able to read the file.

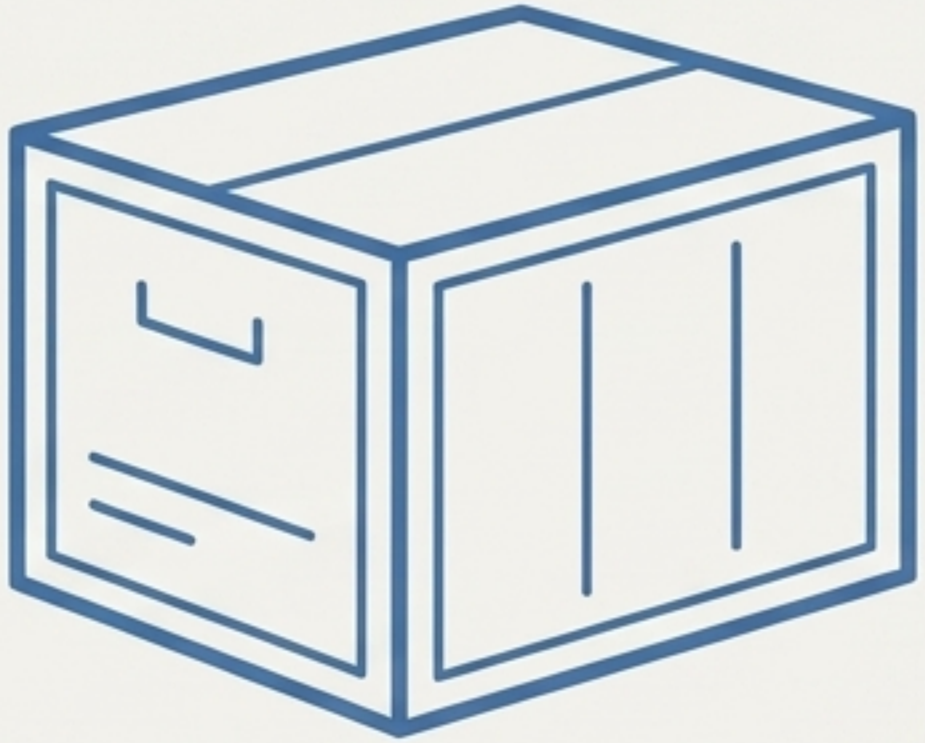
DOCUMENT

- **Not now.**
- **Not if subpoenaed.**
- **Not if breached.**

Every architectural phase, from today to the future, is built to uphold this single rule.

Phase 1: The browser acts as the engine, the server acts as the vault

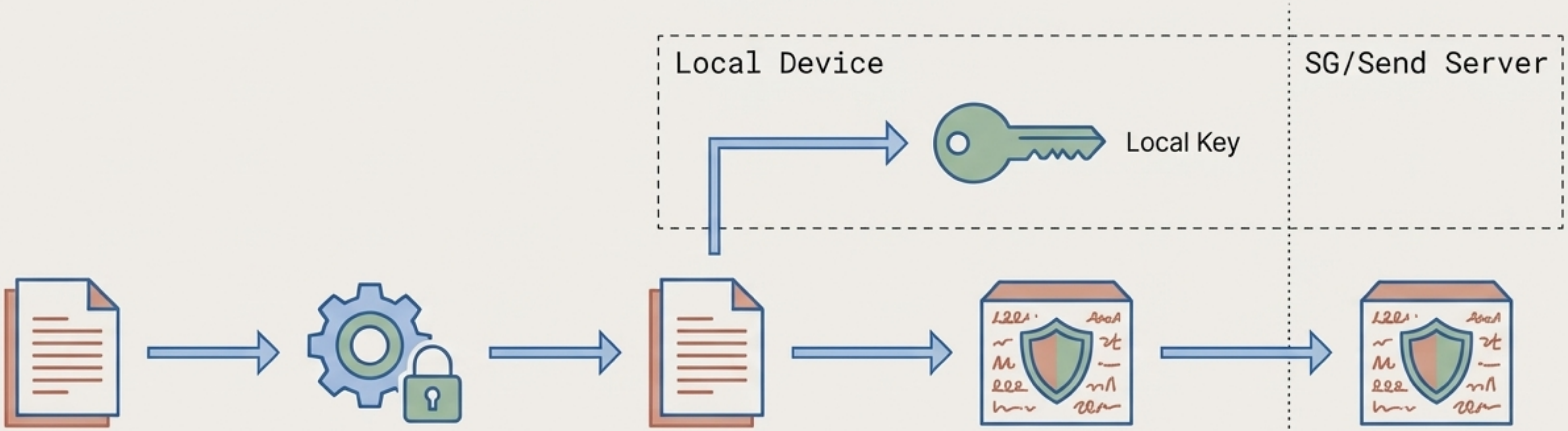
The browser is the encryption engine.
Client Side (Smart)



The server is a dumb blob store.
Server Side (Passive)

The key never touches the server — ever.

The zero-knowledge encryption flow isolates keys from payloads

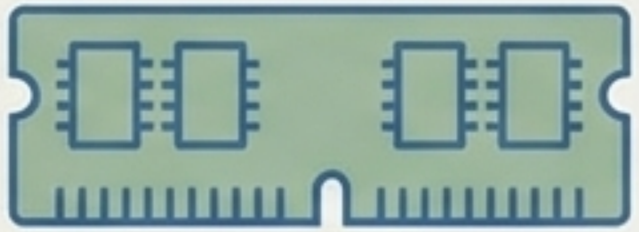


1. File selection in the Browser.
2. Local Key Generation (Client-side).
3. File encrypted into an Encrypted Blob.
4. Decoupling: The key remains local or shared out-of-band, while only the Encrypted Blob moves to the Server.

Architectural routing adapts dynamically for file size

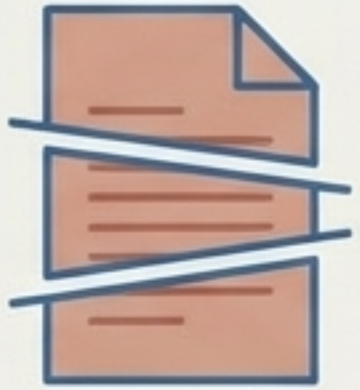
Upload Initiated

Small Files



Processed entirely in memory.
Rapid, single-pass client-side encryption.

Large Files

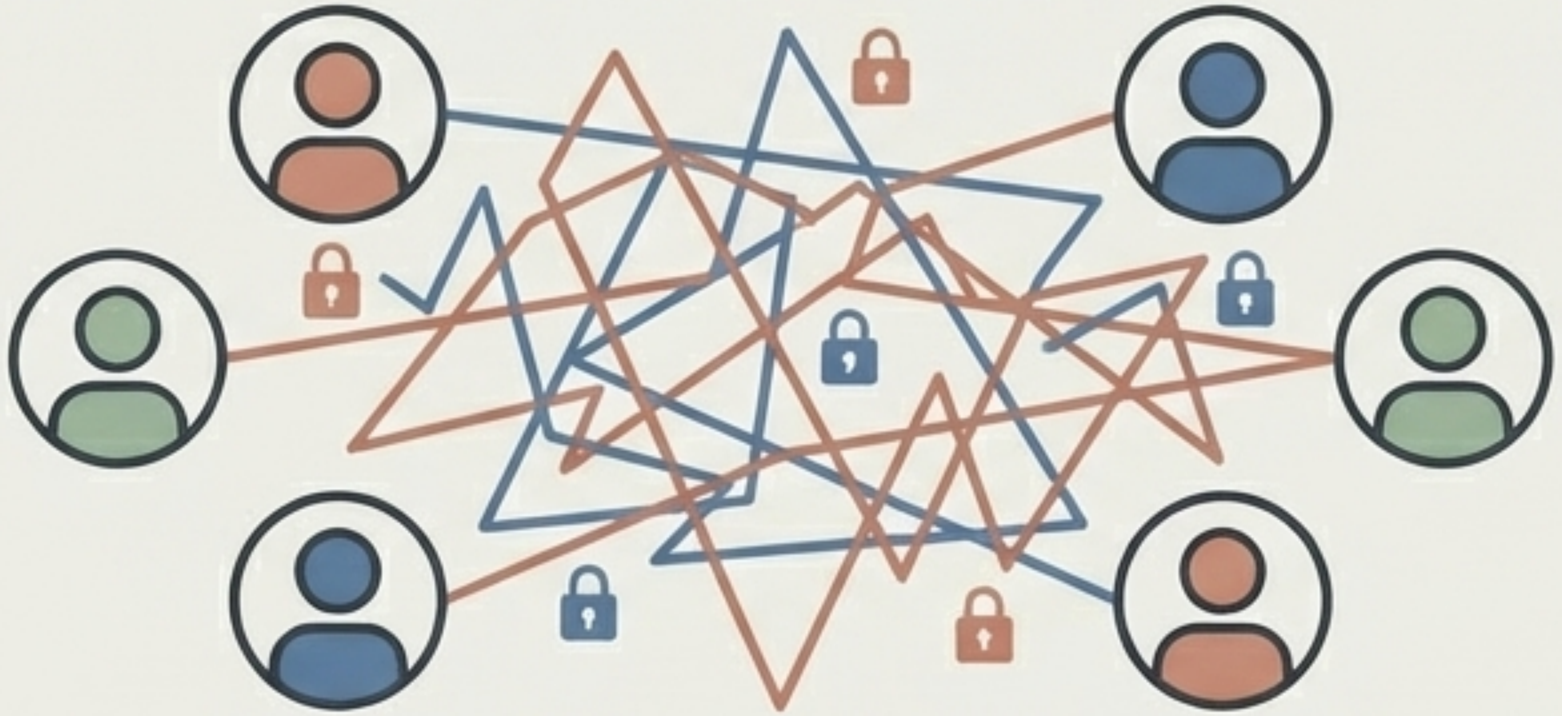


Chunked and streamed encryption.
Bypasses browser memory limits by processing piece by piece.

Both pathways strictly maintain the client-side encryption constraint.

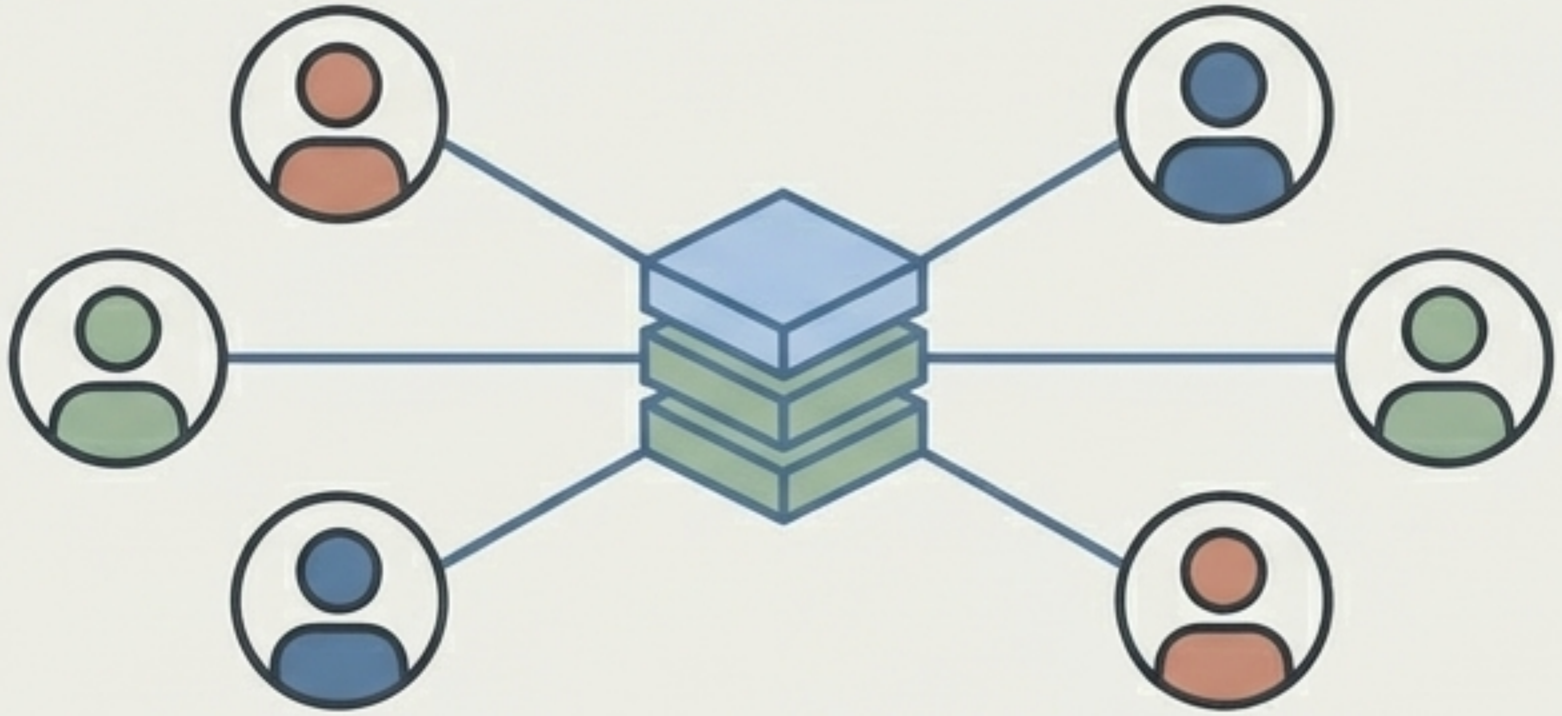
Symmetric keys hit a ceiling in complex enterprise workflows

The Ceiling of Phase 1



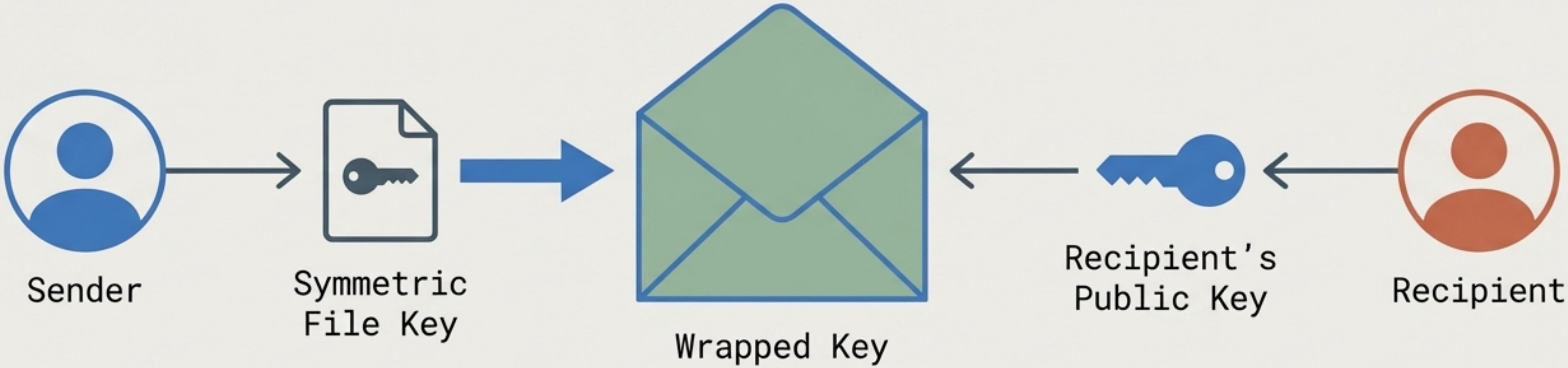
Relying on out-of-band password sharing works perfectly for one-off transfers, but breaks down entirely at enterprise scale, creating massive friction for bidirectional sharing and auditing.

The Solution in Phase 2



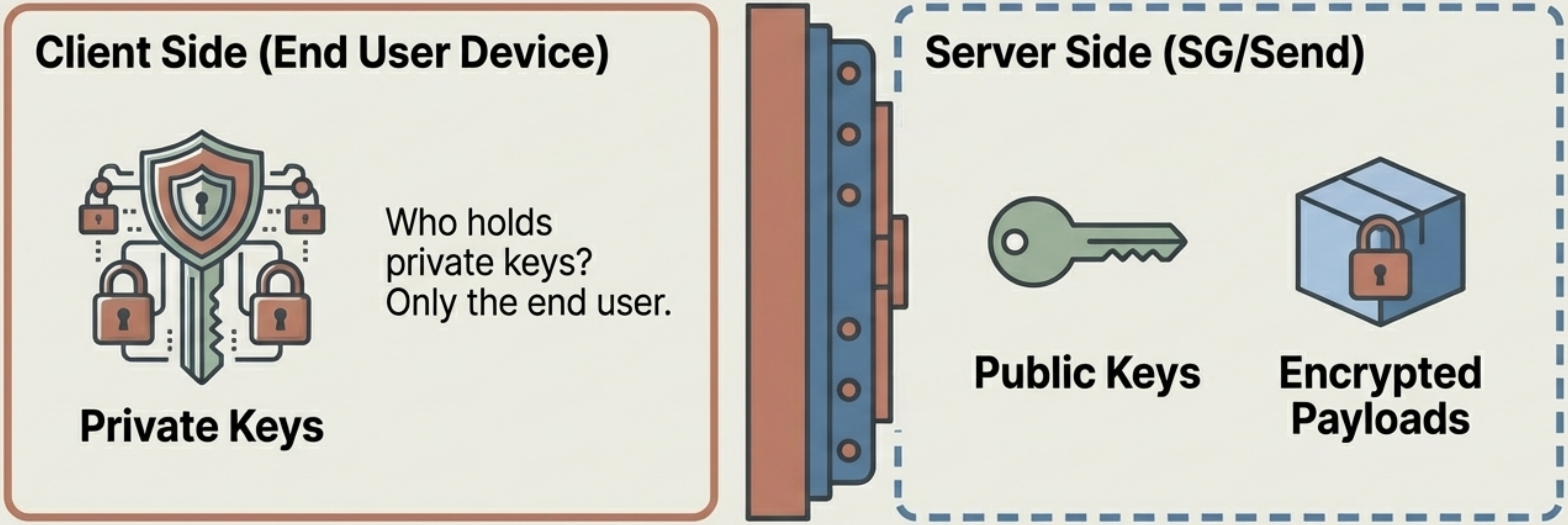
A structured, unified identity system replacing out-of-band symmetric key sharing with scalable, targeted encryption.

Phase 2 introduces a Public Key Infrastructure layer



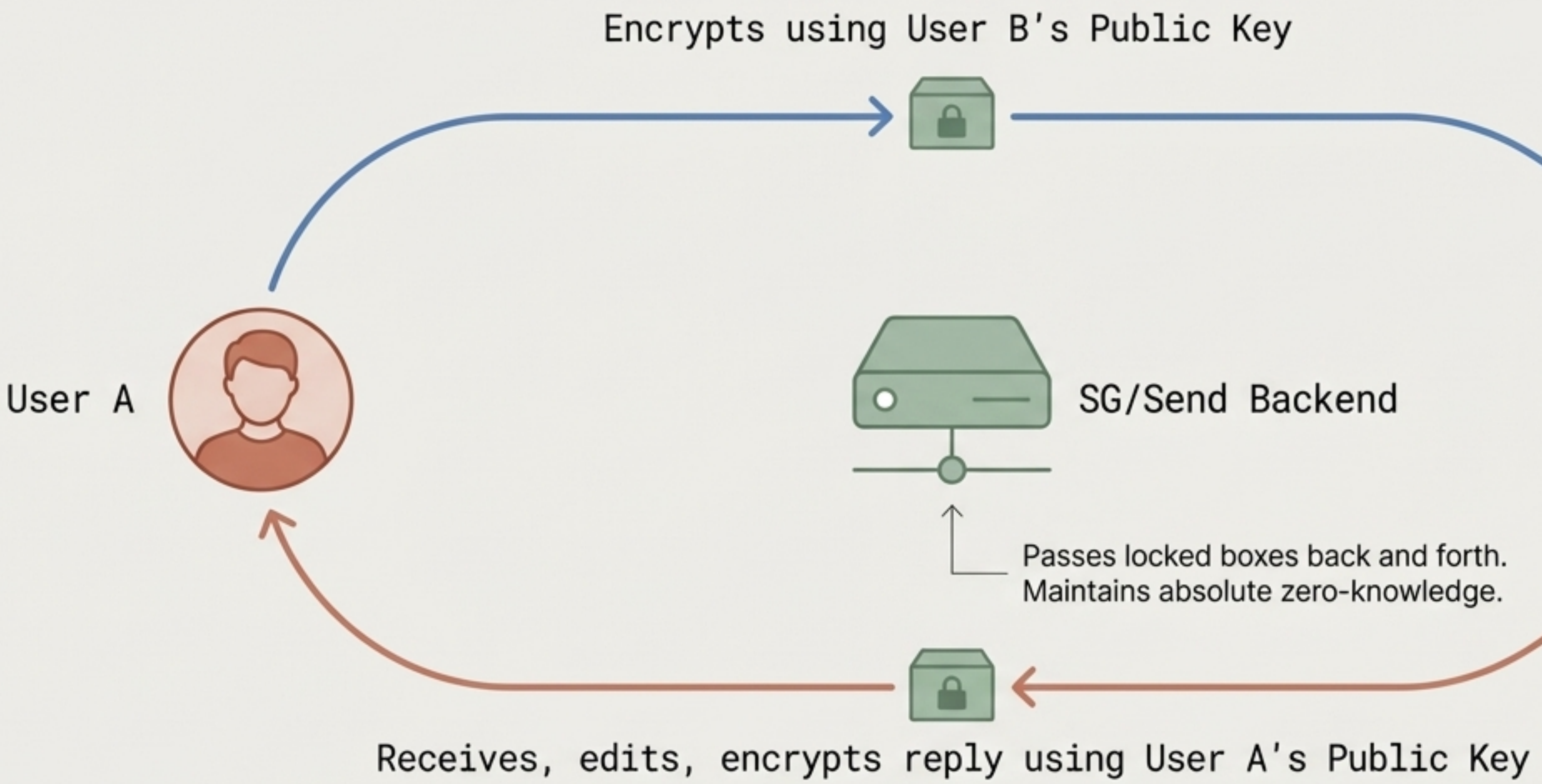
PKI allows users to securely target files to specific recipients using asymmetric key pairs. The sender wraps the symmetric file key with the recipient’s Public Key, completely eliminating the need to share a master password out-of-band, all without exposing the file key to the SG/Send server.

Maintaining zero-knowledge across the PKI trust chain



Even with PKI enabled, the server only ever handles public keys and locked boxes.
The server cannot read the file.

PKI enables frictionless bidirectional document workflows

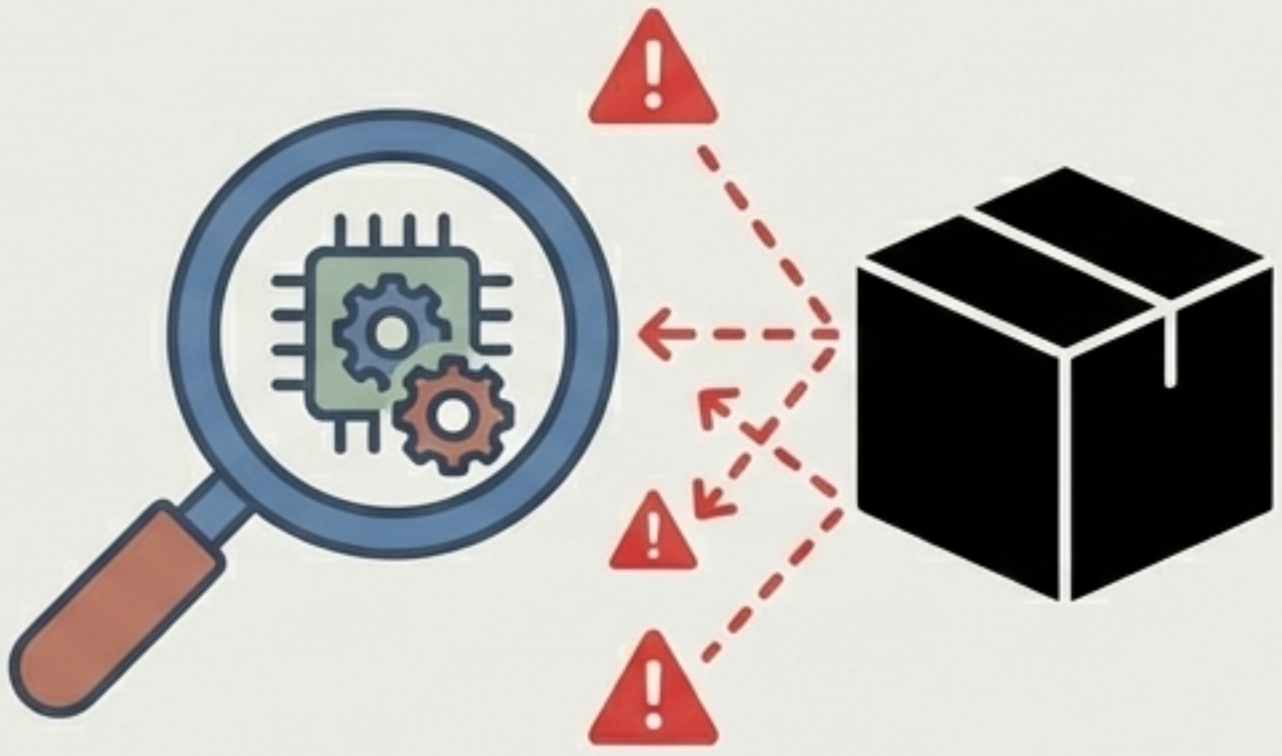


This creates a seamless back-and-forth collaboration environment for enterprises without compromising the core privacy architecture.

The analysis paradox: computing on unreadable data

Perfect Privacy

The Encrypted Blob

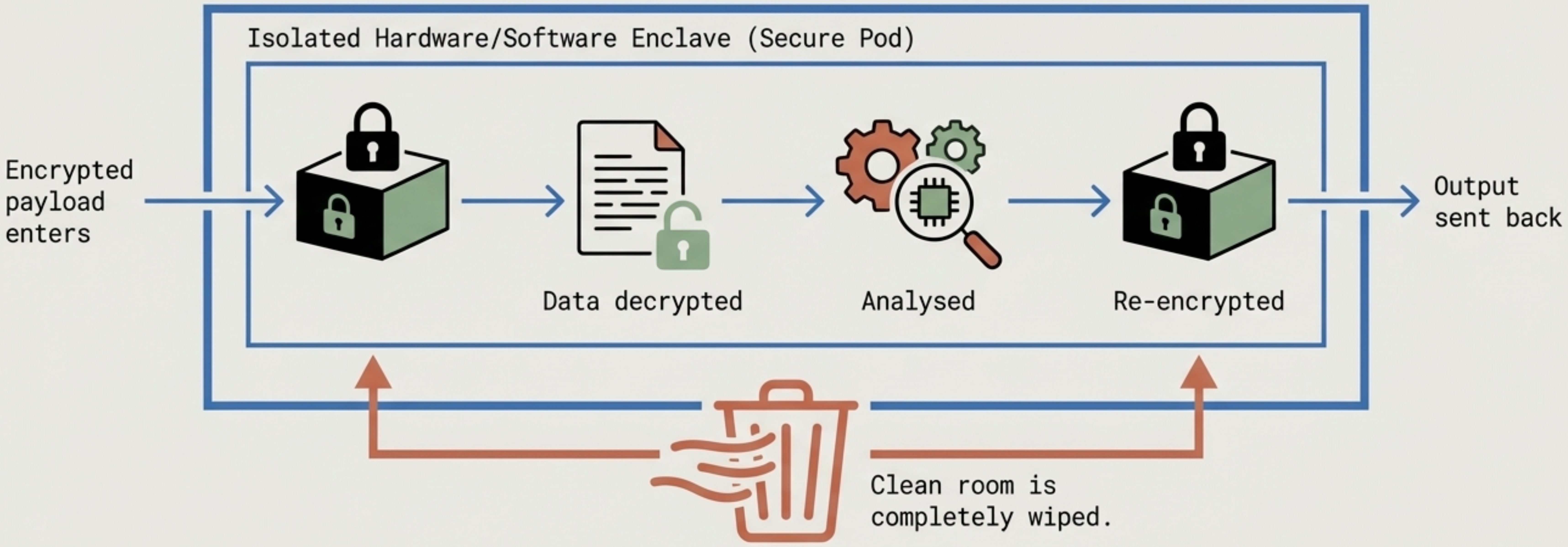


Zero Utility

Cannot scan, search, or analyse.

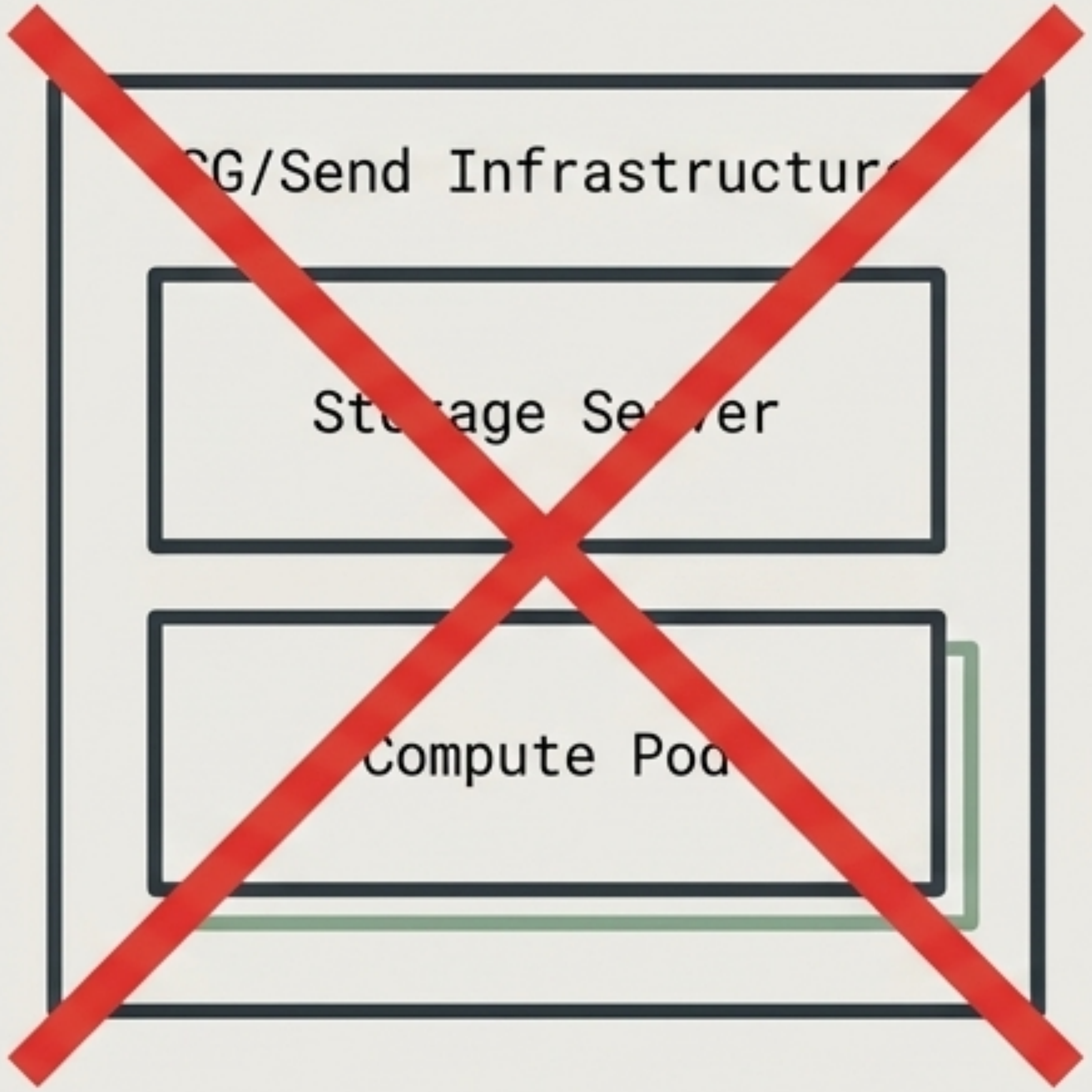
The Problem PKI Alone Doesn’t Solve: How can an enterprise run analytics, AI, or compliance checks on files if no one, not even the server, can read them?

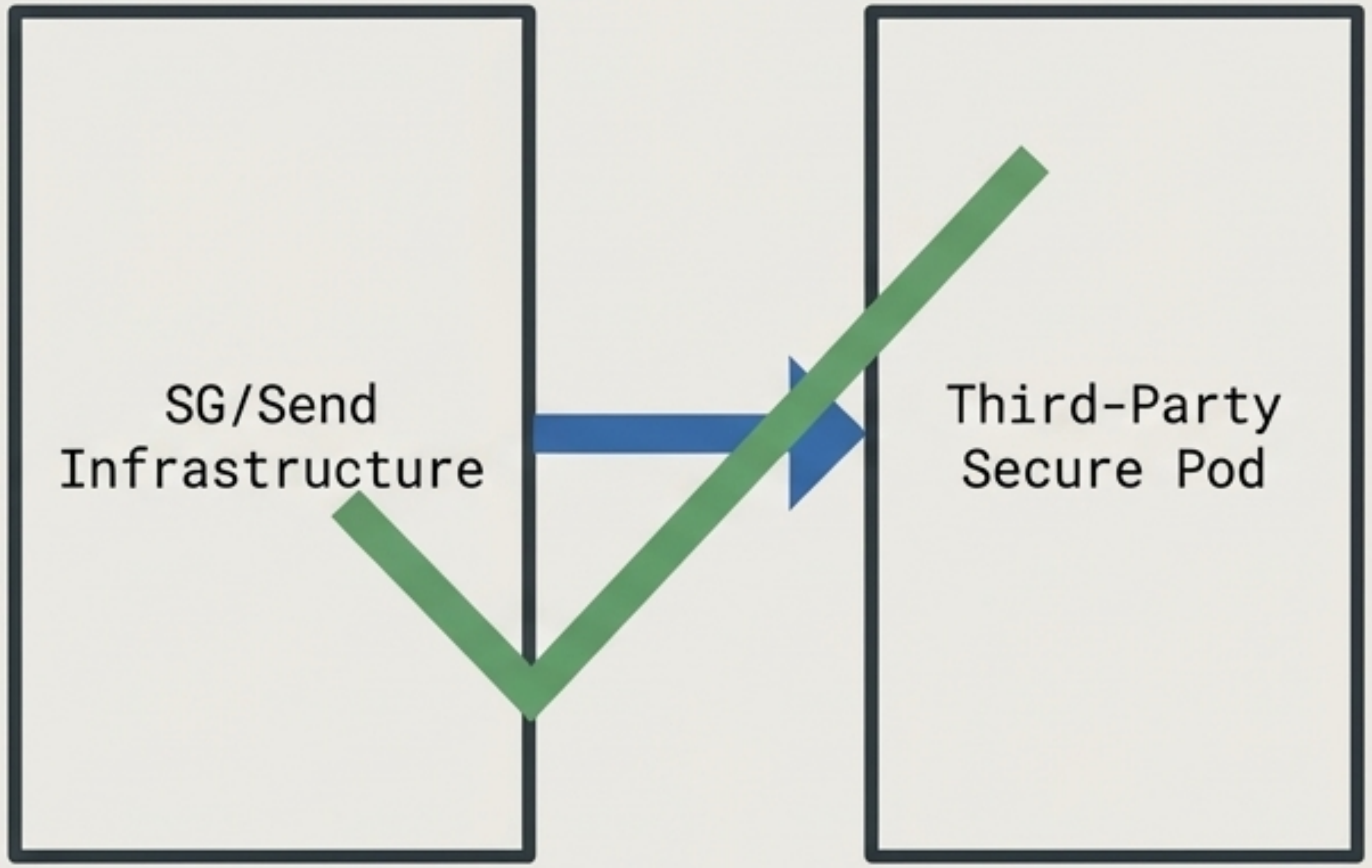
Phase 3: Secure Pods create ephemeral clean rooms for analysis



File analysis without knowing the file. The compute environment is entirely ephemeral and isolated.

Maintaining integrity requires a third-party pod ecosystem





If SG/Send hosts the compute pod, they theoretically have access to the keys and the payload, breaking the zero-knowledge guarantee.

Decentralisation is mandatory. The pod must be a third party to maintain zero-knowledge integrity.

Operational logistics, liability, and charging for compute pods

Actor	Liability Assumed	Cost Centre
End User / Enterprise	Key Management & Access Control	Pays for compute cycles
SG/Send Infrastructure	Encrypted Storage & Routing	Standard Platform Licensing
Third-Party Secure Pod	Data integrity during ephemeral analysis phase	Receives compute revenue

This commercial execution model cleanly separates liability from the zero-knowledge storage layer, creating a scalable, secure ecosystem.

The complete evolutionary architecture of SG/Send

Phase 1: Today	Phase 2: Near-Future	Phase 3: Future
• Core Model • Symmetric Keys • Out-of-band sharing	• PKI Layer • Asymmetric Key Pairs • Bidirectional Workflows	• Secure Pods • Third-Party Enclaves • Zero-Knowledge Compute

Across all phases, the central tenet remains unbroken:
The server never reads the file.

SG/Send is open source (Apache 2.0).
Repository: `https://github.com/the-cyber-boardroom/SGraph-AI__App__Send`
Live platform: `https://send.sgraph.ai`
Note: Secure pod architecture brief [redacted]